# Okay, Now What?

A post-sale toolkit for reps who don't have a customer success team, with Claude working live inside it.

Website: **okay-now-what.vercel.app**

Walkthrough video: **loom.com/share/f07a39bbb16947089f3d2ec074383cdc**

Derek DeCesare · October 2026

Scan to open the site

## The problem

### The rep who closes the deal also owns everything after it.

At GitHub, the mid-market team has no customer success function. The rep who sells Copilot also owns onboarding, adoption, and renewal, on top of a full book and a prospecting number. With large account books at different stages and spend levels, keeping track of everything is hard, and onboarding each account properly isn't realistic.

- Customers get generic docs instead of a plan built for them.
- Champions are left to roll out AI to hundreds of developers on their own.
- Low adoption only shows up at renewal, when it's too late to fix.

> **In my own book:** I own about 40 accounts today, and onboarding one can take multiple meetings, email chains, and follow-ups. This tool sets it up in under 30 seconds. That frees me to focus on finding new business, while the customer gets a guide they can actually use. That naturally increases adoption, usage, and happiness with the tool, which makes it stickier.

## My objective

### Give reps a toolkit that does the work of a CS team, and give every customer a private portal for their whole journey.

HOW I'D MEASURE IT

- **Rep hours on post-sale work:** down, so time goes back to pipeline.
- **Seats active at day 30, 60, and 90:** up, and visible early.
- **Time until the first team is live:** shorter.
- **Renewal and expansion:** protected first, then grown.

## Why it matters

Post-sale work decides whether a deal renews and grows, but today it runs on the rep's spare time.

**For the customer**

- A portal built from their own goals and concerns.
- Learning paths, a champion kit, and easy access to help.
- Answers any time, backed by official docs.

**For the rep**

- The whole book in one place, with risk and growth flagged early.
- Prep a call, log it, and follow up in minutes.
- A new account set up in under a minute.

**For the business**

- Adoption problems show up months before renewal.
- Expansion is spotted from real usage.
- Value the customer's leadership can see.

# How it works

From my deal notes to the customer's whole journey, and back to me.

**1 · REP**

**I add the account**

Paste my deal notes, plus company, seats, and stage.

→

**2 · CLAUDE**

**Builds their portal**

Goals, people, concerns, a dated plan, a setup guide, and a champion kit, in about 25 seconds.

→

**3 · CUSTOMER WORKS THROUGH THEIR JOURNEY**

1 Set up · 2 Learn · 3 Adopt · 4 Prove value · 5 Grow

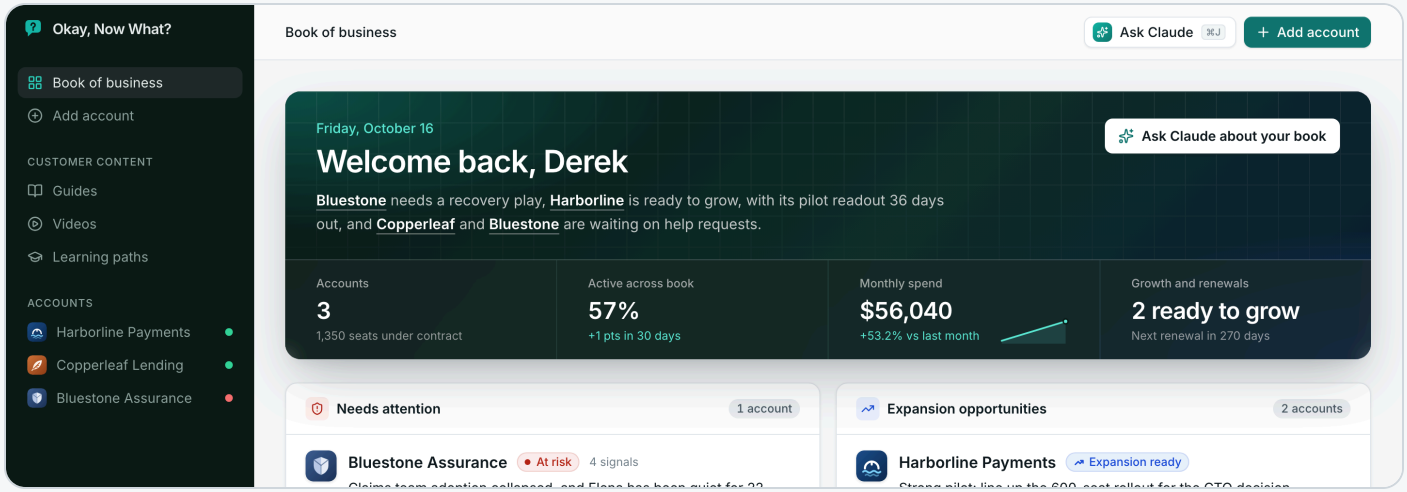
They check off their plan, learn by role, ask questions any time, and get a business review.

**4 · I SEE EVERYTHING, AND CLAUDE HELPS ME ACT ON IT**

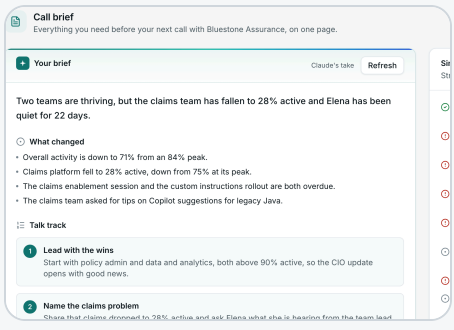
Health and growth signals, adoption, the developer pulse, questions, and help requests. Claude briefs me before calls, updates the account after them, and answers questions about my whole book.

## The rep side

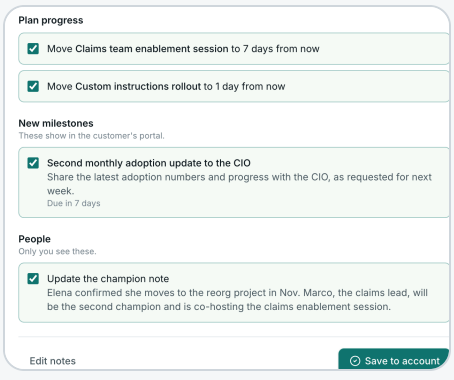
Three fictional accounts show it: Harborline is a pilot ready to grow, Copperleaf is onboarding, and Bluestone is in adoption but at risk.



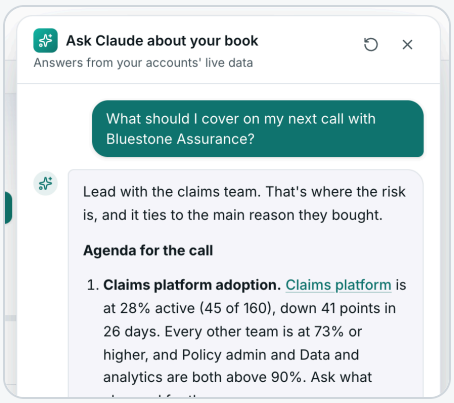

**Book of business.** Where I'd start every morning. It shows who needs help and who's ready to grow, with a suggested play for each. Further down: help requests, customer questions, and deadlines. On each account, **nudges** flag overdue work, slow teams, and quiet champions, with the email already drafted.




**Brief me.** Before a call: what changed, what to say, and what to ask for.



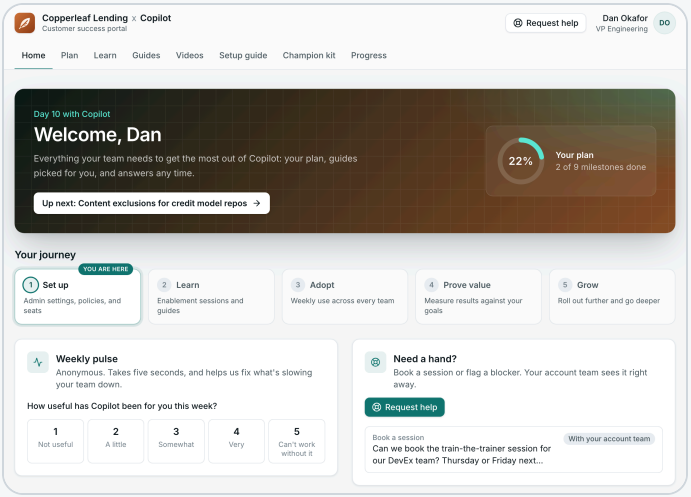

**Log a call.** I paste my notes. Claude proposes the updates and drafts the follow-up, and I pick what to save.



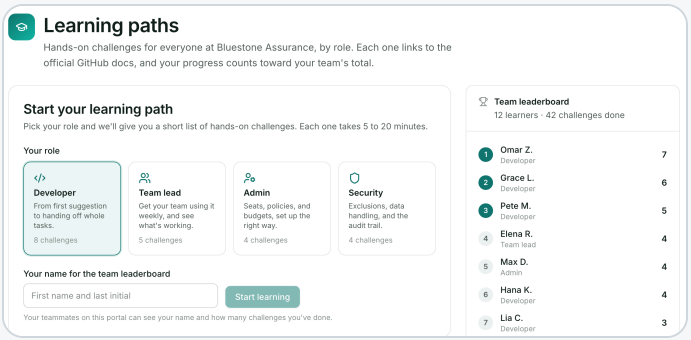

**Ask AI (⌘J).** Questions about one account or the whole book, from any page.

# How it works: the customer side

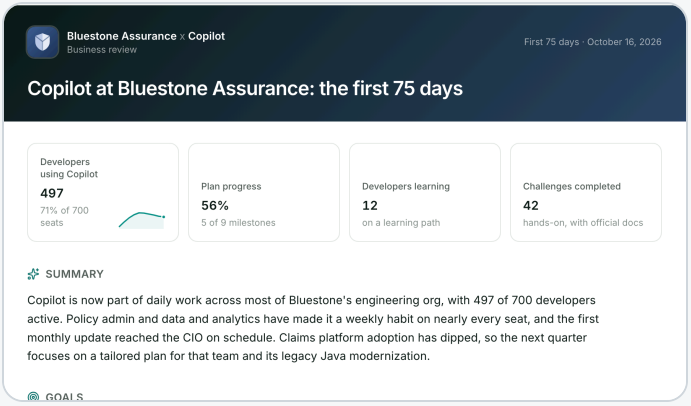
Each customer gets a private link to their own portal. It never shows health, risk, or rep notes, and an automated check scans every page to make sure.



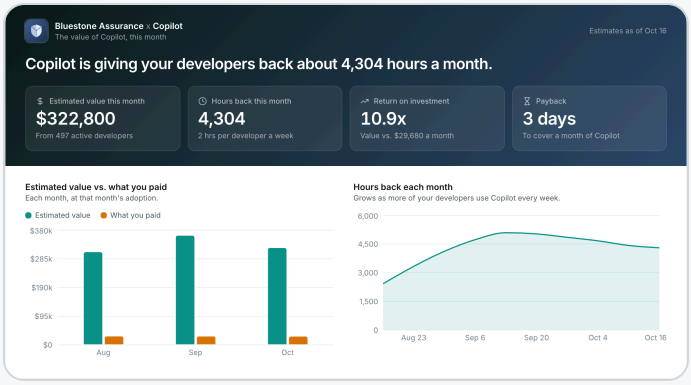

**Portal home.** Their plan, their journey, a quick weekly pulse, and a button to ask us for help.



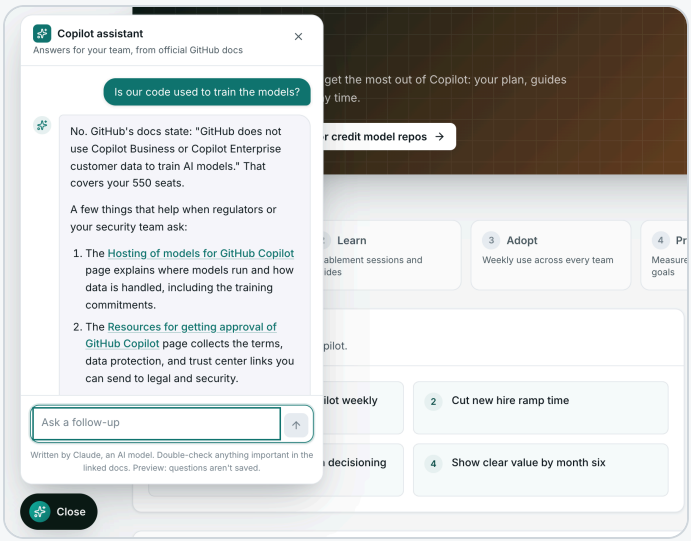

**Learning paths.** Short, hands-on challenges by role, with a team leaderboard.




**Business review.** Written by Claude from what the customer can already see. I check it, then publish it.




**Value.** A one-page report for their leadership: estimated money saved, hours back, ROI, and payback, with charts. I set the assumptions with the customer, and they can print it.




**Ask, on every page.** A Copilot assistant that answers for their company and links to official GitHub docs. I see every question.

**New customer?** I paste my deal notes and Claude builds their whole portal in about 25 seconds, for roughly 7 cents.

### Heads up

Automatic nudges on the home page for anything overdue, due soon, or slow to start.

### First-week checklist

Five small steps, one a day, to get each developer from install to their first Copilot review.

### Prompt recipes

Copy-ready prompts written by Claude for their own codebase, like explaining legacy code or writing tests.

# How I built it

I sell software for a living and had never built an app. With Claude Code I built a live product: about 16,700 lines of code across 179 files. Claude wrote the code; I decided what to build and reviewed every step.

### 1 I planned it in claude.ai

I talked through the problem from my own book, and we turned it into a build spec with nine phases and a test checklist.

### 2 I built it with Claude Code

I gave Claude Code the spec and reviewed each phase in the browser. When it looked too basic, I said so, and it redesigned it.

### 3 I connected Claude to my own app

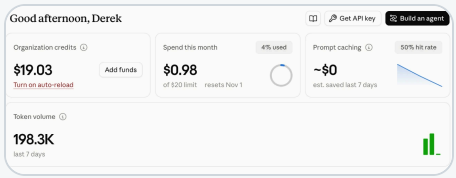
I created an Anthropic API key, set a spending limit, kept the key on the server, and picked Claude Sonnet 5.5 for speed and cost.

> **What I loved:** creating my own API key, so the AI lives inside my app and works live instead of being a static tool.

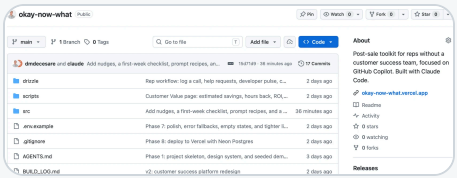
### 4 I put it on the internet

The code is on GitHub and the site runs on Vercel with a Neon database. Claude scanned every commit for secrets before I made it public.

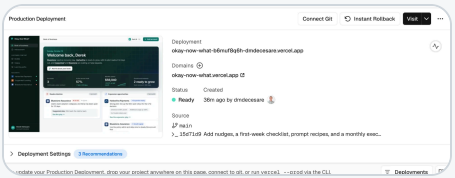
> **What I loved:** being walked through GitHub and Vercel and learning how API keys fit in. It's a real website, not something stuck on my desktop.




**Claude Console.** My API key and a $20 monthly limit.




**GitHub.** The code, with every change saved.




**Vercel.** Runs the site and keeps the key secret.

## How it fits together

Every call to Claude happens on the server.

**BROWSER**

**Rep view**
Book of business
Account tabs, briefs, tools
Ask AI

**Customer view**
Private portal per account
Plan, learning paths, review
Ask assistant

HTTPS →

**VERCEL (SERVER)**

**Next.js server routes**
/api/ask and /api/rep-chat: answers
Server actions: builds, briefs, calls, drafts
Rate limits and input checks

**The API key lives only here,** in a server environment variable. It never reaches the browser.

server to server →

**SERVICES**

**Anthropic API**
Claude Sonnet 5.5
Validated structured outputs
Prompt caching

**Neon Postgres**
Accounts, plans, usage, spend
Questions, calls, help, pulse
Call log for rate limits

## Where Claude runs inside the app

| | |
|---|---|
| **Build account** | Deal notes in, a plan and portal out |
| **Ask assistant** | Customer answers, citing official docs |
| **Ask AI (rep)** | Questions about any account or the whole book |
| **Brief me** | What changed, what to say, what to ask |
| **Log a call** | Notes in, account updates and a follow-up out |
| **Next best action** | 1 to 3 actions from team usage and the plan |
| **Risk play** | Why an account is at risk, and a plan to fix it |
| **Expansion play** | Growth signals turned into a pitch |
| **Question themes** | Customer questions grouped into a summary |
| **Business review** | A review for their leadership |
| **Nudges** | Overdue work flagged, with the email drafted |
| **Prompt recipes** | Copy-ready prompts written for each customer |
| **Drafts** | Launch email, agenda, FAQ, check-in, exec digest |

## The prompts and code behind it

**MY FIRST PROMPT TO CLAUDE CODE**

"Read SPEC.md. Build this app in the phases listed in section 9. Stop after each phase so I can review it in the browser before moving on."

**WHEN THE FIRST VERSION LOOKED BASIC**

"I don't like the UI. I want this to look like a professional website. Imagine you're a real UX engineer building it for a corporate customer who wants something that looks great and real."

**THE CODE THAT CALLS CLAUDE (SRC/LIB/AI/CLIENT.TS, TRIMMED)**

```
const res = await getClient().beta.messages.parse({
  model: cfg.model,
  output_config: { effort: cfg.effort,
    format: betaZodOutputFormat(schema) },
  system, messages: [{ role: "user", content: input }],
});
```

# What I learned

About what Claude can do, about working with it, and about selling it.

## What I learned about Claude's capabilities

- **It plans before it builds.** Each phase started with a plan I could approve, and it caught two contradictions in my spec.
- **It checks its own work.** It wrote a test to keep internal data off customer pages, and that test later caught a real mistake.
- **It notices things I didn't ask about.** It found that GitHub had changed Copilot's billing terms, and that my previews of the customer portal were being saved as if the customer had asked.
- **It covers the whole stack.** Database, design, prompts, security checks, and deployment, all in one conversation.
- **It works inside the product, not just on it.** Claude answers customers, writes briefs, and updates accounts live. That's what turns this from a static tool into something I'd use every day.

## What surprised me

How much Claude Code could connect to. It walked me through creating a Vercel account, an Anthropic organization API key, and a GitHub repo, showed me how they all fit together, and ended with a website I can actually use and send out. It also added value beyond the code by suggesting ideas and helping me set up agents to work alongside me. And I didn't expect how much it could do on my laptop: it installed Git and other tools and worked across my systems.

## What I learned about prompting

Putting lots of detail and several changes into one message was much cheaper and more efficient than sending small changes one at a time. That mattered more as the project grew and every message carried more context, and it was crucial because I was using up credits quickly.

## Where Claude struggled, and how we got past it

- **It copied a line of my deal notes into a customer page.** Its own leak check caught it, and we added a "who sees what" rule.
- **The first redesign still looked basic to me.** I told it so, and the second pass looked like a real product.
- **Log a call first created duplicate milestones.** Now it reschedules overdue work instead of copying it.
- **Big context drained tokens.** As the project grew, every message carried the full context, so each step took longer and cost more.

## What this means for selling Claude

**You don't have to be an engineer.**
I shipped a live product with a database, an API, and a deploy. That widens who a Claude Code deal is for.

**Adoption is the renewal.**
Seats and usage only grow when people use the product. Post-sale is where that revenue is won or lost.

**Lead with the guardrails.**
The leak checks, the secrets scan, and the tested failure paths are what a CISO will ask about first.

**Cost is a choice you can explain.**
A fast model for answers and more effort for builds and reviews: about seven cents per account built.

## What's next

### Real usage and billing data

Pull in GitHub's Copilot usage and billing data, so health, growth, and spend update on their own.

### CRM integration

Pull deal notes and contacts from the CRM when a deal closes, and push health and call notes back.

### The same model for Claude

Point it at Claude Enterprise and Claude Code rollouts: seats, a journey, learning paths, and Anthropic's docs.

# How to use it

You can try both sides in about five minutes.

### Get in

Open **okay-now-what.vercel.app** or scan the code on page 1. You land on the rep side as me. No login needed.

### See the customer side

Each customer has a private link. From any account, click **Open customer view** to see their portal.

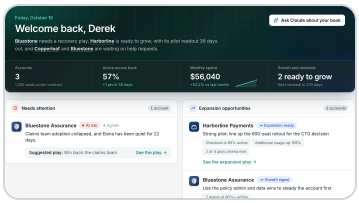
### Click around freely

In preview, nothing you click counts as the customer. All companies and data are fictional.

## REP Running my book

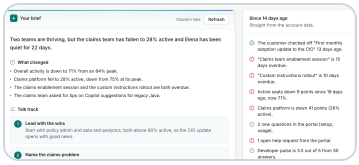
### 1 Read the week

The summary at the top says who needs a recovery play, who's ready to grow, and what's waiting on you.



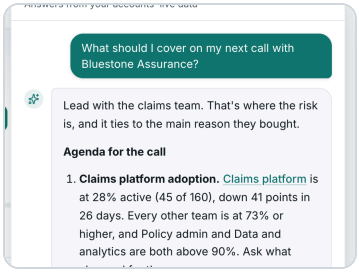

### 2 Open Bluestone and click Brief me

A one-page brief: what changed, what to say, and what to ask for.



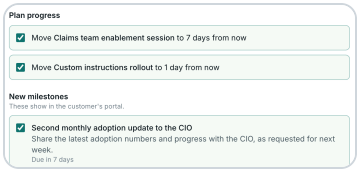

### 3 Press ⌘J and ask a question

Try "Which accounts need me this week?"




### 4 Click Log a call

Paste a few notes, tick the updates you want, and copy the follow-up email.




### 5 Click Open customer view

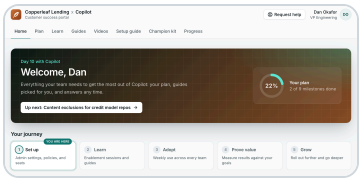
See that account's portal exactly as the customer does.




## CUSTOMER Onboarding my team

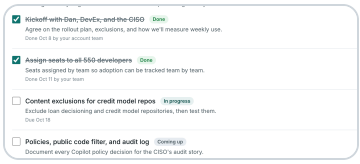
### 1 Start on the portal home

Your journey, goals, plan progress, and what to do this week.



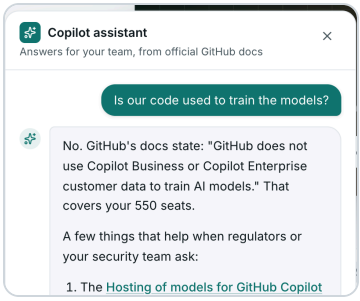

### 2 Check off your plan

Tick a milestone on **Plan**. Your account team sees it right away.



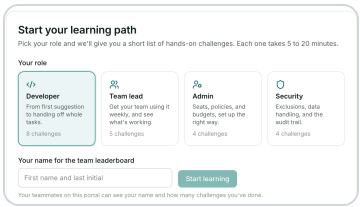

### 3 Ask anything, from any page

Use the **Ask about Copilot** button in the corner.




### 4 Pick a role on Learn

Short hands-on challenges, with a team leaderboard.




### 5 See your value, or ask for help

Open **Value** to see savings and ROI. Answer the pulse or use **Request help** any time.

